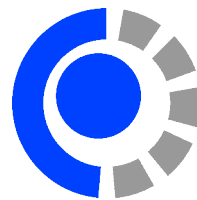




UNIVERSIDAD NACIONAL DEL COMAHUE
FACULTAD DE INFORMÁTICA



TRABAJO DE LABORATORIO N°4

DISEÑO DE COMPILADORES E INTÉRPRETES

Grupo 5

Franco Fabris

franco.fabris@est.fi.uncoma.edu.ar

FAI-3206

Valentina Villarroel

valentina.villarroel@est.fi.uncoma.edu.ar

FAI-3944

NEUQUÉN

ARGENTINA

2026

Contents

1	Descripción del problema	3
2	Estrategias utilizadas	3
3	Limitaciones	3
4	Modificación de la Gramática	4
4.1	Transformación a BNF	4
4.2	Eliminando la Recursión a Izquierda	6
4.3	Eliminando la Factorización	6
5	Gramática de tipo LL(1)	12
6	Gramática final	12
7	Analizador Sintáctico Descendente Recursivo Predictivo	15
8	Problemas encontrados	25
9	Conclusión	25

1 Descripción del problema

El objetivo de este trabajo de laboratorio es diseñar e implementar un Analizador Sintáctico para un subconjunto del lenguaje Pascal. Partiendo de la gramática definida en los trabajos anteriores, el desafío consiste en transformarla en una gramática de tipo LL(1) que sea apta para un análisis descendente predictivo.

La tarea principal del analizador sintáctico es recibir la secuencia de componentes léxicos (tokens) generados por el scanner y verificar que respeten las reglas estructurales del lenguaje. Para ello, debemos resolver conflictos gramaticales comunes como la recursión a izquierda y la ambigüedad en los prefijos, asegurando que el parser pueda tomar decisiones de derivación utilizando únicamente un símbolo de preanálisis, el cual nombraremos como lookahead de acá en adelante.

2 Estrategias utilizadas

Para la resolución de este laboratorio se aplicaron las siguientes estrategias:

1. **Transformación de la Gramática:** Se analizaron las reglas originales para detectar recursión a izquierda.
2. **Factorización por Izquierda:** Se identificaron reglas donde múltiples alternativas comenzaban con el mismo token.
3. **Resolución de la Ambigüedad:** Se forzó la asociación del bloque `else` al `if` más interno para cumplir con la semántica de Pascal.
4. **Implementación Analizador Sintáctico:** Se diseñó de forma tal que cada no terminal de la gramática se corresponde con un módulo en el código. Se sigue con el estándar introducido por el Analizador Léxico.

3 Limitaciones

El diseño actual del compilador presenta las siguientes restricciones:

- **Recuperación de errores simple:** El parser está diseñado para detener la ejecución y reportar el primer error encontrado. No se han implementado estrategias avanzadas de recuperación que permitan continuar el análisis tras detectar una falla.
- **Gramática Estricta:** No se admiten construcciones fuera del subconjunto de Pascal definido, como punteros, registros o tipos de datos reales.

4 Modificación de la Gramática

4.1 Transformación a BNF

$\langle statement \rangle ::= \text{'program'} \langle ident \rangle \text{';} \langle bloque \rangle \text{'.'}$

$\langle bloque \rangle ::= \text{'var'} \langle sentencia_de_tipos \rangle \langle lista_subprogramas \rangle \langle cuerpo \rangle$
| $\langle lista_subprogramas \rangle \langle cuerpo \rangle$
| $\langle cuerpo \rangle$

$\langle sentencia_de_tipos \rangle ::= \langle declaración_de_var \rangle \text{';} \langle sentencia_de_tipos \rangle$
| $\langle declaración_de_var \rangle \text{';'}$

$\langle declaración_de_var \rangle ::= \langle lista_var \rangle \text{'::'} \langle tipo \rangle$

$\langle lista_var \rangle ::= \langle ident \rangle \text{' ,' } \langle lista_var \rangle$
| $\langle ident \rangle$

$\langle tipo \rangle ::= \text{'integer'} \mid \text{'boolean'}$

$\langle lista_subprogramas \rangle ::= \langle subprograma \rangle \text{';} \langle lista_subprogramas \rangle$
| $\langle subprograma \rangle$

$\langle subprograma \rangle ::= \langle funcion \rangle \mid \langle procedimiento \rangle$

$\langle funcion \rangle ::= \text{'function'} \langle ident \rangle \text{'(' } \langle param_formales \rangle \text{')' ' '::' } \langle tipo \rangle \text{';' } \langle bloque \rangle$
| $\text{'function'} \langle ident \rangle \text{' '::' } \langle tipo \rangle \text{';' } \langle bloque \rangle$

$\langle procedimiento \rangle ::= \text{'procedure'} \langle ident \rangle \text{';' } \langle bloque \rangle$
| $\text{'procedure'} \langle ident \rangle \text{'(' } \langle param_formales \rangle \text{')' ' '::' } \langle bloque \rangle$

$\langle param_formales \rangle ::= \langle lista_ident \rangle \text{' '::' } \langle tipo \rangle$
| $\langle lista_ident \rangle \text{' '::' } \langle tipo \rangle \text{';' } \langle param_formales \rangle$

$\langle cuerpo \rangle ::= \text{'begin'} \langle lista_sentencias \rangle \text{'end'}$

$\langle lista_sentencias \rangle ::= \langle sentencia \rangle \text{';' } \langle lista_sentencias \rangle$
| $\langle sentencia \rangle$

$\langle sentencia \rangle ::= \langle asignación \rangle$
| $\langle llamada_procedimiento \rangle$
| $\langle alternativa \rangle$
| $\langle repetitiva \rangle$
| $\langle cuerpo \rangle$

$\langle lectura \rangle$
 $\mid$
 $\langle escritura \rangle$

$\langle asignación \rangle ::= \langle ident \rangle \text{' := ' } \langle expresion \rangle$

$\langle llamada_procedimiento \rangle ::= \langle ident \rangle$
 $\mid \langle ident \rangle \text{' (' } \langle lista_expresion \rangle \text{') '}$

$\langle alternativa \rangle ::= \text{' if ' } \langle expresion \rangle \text{' then ' } \langle sentencia \rangle \text{' else ' } \langle sentencia \rangle$
 $\mid \text{' if ' } \langle expresion \rangle \text{' then ' } \langle sentencia \rangle$

$\langle repetitiva \rangle ::= \text{' while ' } \langle expresion \rangle \text{' do ' } \langle sentencia \rangle$

$\langle lectura \rangle ::= \text{' read ' } \text{' (' } \langle ident \rangle \text{') '}$

$\langle escritura \rangle ::= \text{' write ' } \text{' (' } \langle expresion \rangle \text{') '}$

$\langle lista_expresion \rangle ::= \langle expresion \rangle \text{' , ' } \langle lista_expresion \rangle$
 $\mid \langle expresion \rangle$

$\langle expresion \rangle ::= \langle expresion_simple \rangle \langle relacion \rangle \langle expresion_simple \rangle$
 $\mid \langle expresion_simple \rangle$

$\langle relacion \rangle ::= \text{' = ' } \mid \text{' < > ' } \mid \text{' < ' } \mid \text{' < = ' } \mid \text{' > ' } \mid \text{' > = '}$

$\langle expresion_simple \rangle ::= \langle termino \rangle$
 $\mid \langle signo \rangle \langle termino \rangle$
 $\mid \langle expresion_simple \rangle \langle lista_expresion_simple \rangle$

$\langle lista_expresion_simple \rangle ::= \text{' or ' } \langle termino \rangle$
 $\mid \langle signo \rangle \langle termino \rangle$

$\langle signo \rangle ::= \text{' + ' } \mid \text{' - '}$

$\langle termino \rangle ::= \langle factor \rangle$
 $\mid \langle factor \rangle \langle lista_terminos \rangle$

$\langle lista_terminos \rangle ::= \langle operacion \rangle \langle factor \rangle$
 $\mid \langle operacion \rangle \langle factor \rangle \langle lista_terminos \rangle$

$\langle operacion \rangle ::= \text{' * ' } \mid \text{' / ' } \mid \text{' and '}$

$$\begin{aligned} \langle factor \rangle &::= \langle ident \rangle \\ &| \langle numero \rangle \\ &| \text{'true'} \\ &| \text{'false'} \\ &| \langle llamada_funcion \rangle \\ &| \text{'('} \langle expresion \rangle \text{'})'} \\ &| \text{'not'} \langle factor \rangle \end{aligned}$$

$$\begin{aligned} \langle llamada_funcion \rangle &::= \langle ident \rangle \\ &| \langle ident \rangle \text{'('} \langle lista_expresion \rangle \text{'})'} \end{aligned}$$

$$\begin{aligned} \langle ident \rangle &::= \langle letra \rangle \\ &| \langle letra \rangle \langle lista_ident \rangle \end{aligned}$$

$$\begin{aligned} \langle lista_ident \rangle &::= \langle letra \rangle | \langle digito \rangle \\ &| \langle letra \rangle \langle lista_ident \rangle \\ &| \langle digito \rangle \langle lista_ident \rangle \end{aligned}$$

$$\begin{aligned} \langle letra \rangle &::= \text{'a'} | \text{'b'} | \text{'c'} | \text{'d'} | \text{'e'} | \text{'f'} | \text{'g'} | \text{'h'} | \text{'i'} | \text{'j'} | \text{'k'} | \text{'l'} | \text{'m'} | \text{'n'} | \text{'o'} | \text{'p'} | \text{'q'} | \text{'r'} | \text{'s'} | \\ &\quad \text{'t'} | \text{'u'} | \text{'v'} | \text{'w'} | \text{'x'} | \text{'y'} | \text{'z'} \end{aligned}$$

$$\begin{aligned} \langle numero \rangle &::= \langle digito \rangle \langle numero \rangle \\ &| \langle digito \rangle \end{aligned}$$

$$\langle digito \rangle ::= \text{'0'} | \text{'1'} | \text{'2'} | \text{'3'} | \text{'4'} | \text{'5'} | \text{'6'} | \text{'7'} | \text{'8'} | \text{'9'}$$

4.2 Eliminando la Recursión a Izquierda

$$\begin{aligned} \langle expresion_simple \rangle &::= \langle termino \rangle \\ &| \langle signo \rangle \langle termino \rangle \\ &| \langle expresion_simple \rangle \langle lista_expresion_simple \rangle \end{aligned}$$

Pasa a ser

$$\begin{aligned} \langle expresion_simple \rangle &::= \langle termino \rangle \langle expresion_simple_prima \rangle \\ &| \langle signo \rangle \langle termino \rangle \langle expresion_simple_prima \rangle \end{aligned}$$

$$\langle expresion_simple_prima \rangle ::= \langle lista_expresion_simple \rangle \langle expresion_simple_prima \rangle | \lambda$$

4.3 Eliminando la Factorización

$$\begin{aligned} \langle sentencia_de_tipos \rangle &::= \langle declaración_de_var \rangle \text{';' } \langle sentencia_de_tipos \rangle \\ &| \langle declaración_de_var \rangle \text{';' } \end{aligned}$$

Pasa a ser

$\langle \text{sentencia_de_tipos} \rangle ::= \langle \text{declaración_de_var} \rangle \text{' ; ' } \langle \text{sentencia_de_tipos_prima} \rangle$

$\langle \text{sentencia_de_tipos_prima} \rangle ::= \langle \text{sentencia_de_tipos} \rangle$
 $\quad | \quad \lambda$

$\langle \text{lista_var} \rangle ::= \langle \text{ident} \rangle \text{' , ' } \langle \text{lista_var} \rangle$
 $\quad | \quad \langle \text{ident} \rangle$

Pasa a ser

$\langle \text{lista_var} \rangle ::= \langle \text{ident} \rangle \text{' , ' } \langle \text{lista_var_prima} \rangle$

$\langle \text{lista_var_prima} \rangle ::= \langle \text{lista_var} \rangle$
 $\quad | \quad \lambda$

$\langle \text{lista_subprogramas} \rangle ::= \langle \text{subprograma} \rangle \text{' ; ' } \langle \text{lista_subprogramas} \rangle$
 $\quad | \quad \langle \text{subprograma} \rangle$

Pasa a ser

$\langle \text{lista_subprogramas} \rangle ::= \langle \text{subprograma} \rangle \langle \text{lista_subprogramas_prima} \rangle$

$\langle \text{lista_subprogramas_prima} \rangle ::= \text{' ; ' } \langle \text{lista_subprogramas} \rangle$
 $\quad | \quad \lambda$

$\langle \text{funcion} \rangle ::= \text{' function ' } \langle \text{ident} \rangle \text{' (' } \langle \text{param_formales} \rangle \text{') ' } \text{' : ' } \langle \text{tipo} \rangle \text{' ; ' } \langle \text{bloque} \rangle$
 $\quad | \quad \text{' function ' } \langle \text{ident} \rangle \text{' : ' } \langle \text{tipo} \rangle \text{' ; ' } \langle \text{bloque} \rangle$

Pasa a ser

$\langle \text{funcion} \rangle ::= \text{' function ' } \langle \text{ident} \rangle \langle \text{funcion_prima} \rangle$

$\langle \text{funcion_prima} \rangle ::= \text{' (' } \langle \text{param_formales} \rangle \text{') ' } \text{' : ' } \langle \text{tipo} \rangle \text{' ; ' } \langle \text{bloque} \rangle$
 $\quad | \quad \text{' : ' } \langle \text{tipo} \rangle \text{' ; ' } \langle \text{bloque} \rangle$

$$\langle \text{procedimiento} \rangle ::= \text{'procedure'} \langle \text{ident} \rangle \text{' ; ' } \langle \text{bloque} \rangle$$

$$| \text{'procedure'} \langle \text{ident} \rangle \text{' (' } \langle \text{param_formales} \rangle \text{') ' ' ; ' } \langle \text{bloque} \rangle$$

Pasa a ser

$$\langle \text{procedimiento} \rangle ::= \text{'procedure'} \langle \text{ident} \rangle \langle \text{procedimiento_prima} \rangle$$

$$\langle \text{procedimiento_prima} \rangle ::= \text{' (' } \langle \text{param_formales} \rangle \text{') ' ' ; ' } \langle \text{bloque} \rangle$$

$$| \text{' ; ' } \langle \text{bloque} \rangle$$

$$\langle \text{param_formales} \rangle ::= \langle \text{lista_ident} \rangle \text{' : ' } \langle \text{tipo} \rangle$$

$$| \langle \text{lista_ident} \rangle \text{' : ' } \langle \text{tipo} \rangle \text{' ; ' } \langle \text{param_formales} \rangle$$

Pasa a ser

$$\langle \text{param_formales} \rangle ::= \langle \text{lista_ident} \rangle \text{' : ' } \langle \text{tipo} \rangle \langle \text{param_formales_prima} \rangle$$

$$\langle \text{param_formales_prima} \rangle ::= \text{' ; ' } \langle \text{param_formales} \rangle$$

$$| \lambda$$

$$\langle \text{lista_sentencias} \rangle ::= \langle \text{sentencia} \rangle \text{' ; ' } \langle \text{lista_sentencias} \rangle$$

$$| \langle \text{sentencia} \rangle$$

Pasa a ser

$$\langle \text{lista_sentencias} \rangle ::= \langle \text{sentencia} \rangle \langle \text{lista_sentencias_prima} \rangle$$

$$\langle \text{lista_sentencias_prima} \rangle ::= \text{' ; ' } \langle \text{lista_sentencias} \rangle$$

$$| \lambda$$

$$\langle \text{sentencia} \rangle ::= \langle \text{asignación} \rangle$$

$$| \langle \text{llamada_procedimiento} \rangle$$

$$| \langle \text{alternativa} \rangle$$

$$| \langle \text{repetitiva} \rangle$$

$$| \langle \text{cuerpo} \rangle$$

$$| \langle \text{lectura} \rangle$$

$$| \langle \text{escritura} \rangle$$

$$\langle \text{asignación} \rangle ::= \langle \text{ident} \rangle \text{' := ' } \langle \text{expresion} \rangle$$

$$\langle \text{llamada_procedimiento} \rangle ::= \langle \text{ident} \rangle$$

$$| \langle \text{ident} \rangle ' (' \langle \text{lista_expresion} \rangle ')'$$

Pasa a ser

$$\langle \text{sentencia} \rangle ::= \langle \text{ident} \rangle \langle \text{sentencia_prima} \rangle$$

$$| \langle \text{alternativa} \rangle$$

$$| \langle \text{repetitiva} \rangle$$

$$| \langle \text{cuerpo} \rangle$$

$$| \langle \text{lectura} \rangle$$

$$| \langle \text{escritura} \rangle$$

$$\langle \text{sentencia_prima} \rangle ::= ' := ' \langle \text{expresion} \rangle$$

$$| ' (' \langle \text{lista_expresion} \rangle ')'$$

$$| \lambda$$

$$\langle \text{alternativa} \rangle ::= ' \text{if} ' \langle \text{expresion} \rangle ' \text{then} ' \langle \text{sentencia} \rangle ' \text{else} ' \langle \text{sentencia} \rangle$$

$$| ' \text{if} ' \langle \text{expresion} \rangle ' \text{then} ' \langle \text{sentencia} \rangle$$

Pasa a ser

$$\langle \text{alternativa} \rangle ::= ' \text{if} ' \langle \text{expresion} \rangle ' \text{then} ' \langle \text{sentencia} \rangle \langle \text{alternativa_prima} \rangle$$

$$\langle \text{alternativa_prima} \rangle ::= ' \text{else} ' \langle \text{sentencia} \rangle$$

$$| \lambda$$

$$\langle \text{lista_expresion} \rangle ::= \langle \text{expresion} \rangle ' , ' \langle \text{lista_expresion} \rangle$$

$$| \langle \text{expresion} \rangle$$

Pasa a ser

$$\langle \text{lista_expresion} \rangle ::= \langle \text{expresion} \rangle \langle \text{lista_expresion_prima} \rangle$$

$$\langle \text{lista_expresion_prima} \rangle ::= ' , ' \langle \text{lista_expresion} \rangle$$

$$| \lambda$$

$$\langle \text{expresion} \rangle ::= \langle \text{expresion_simple} \rangle \langle \text{relacion} \rangle \langle \text{expresion_simple} \rangle$$

$$| \langle \text{expresion_simple} \rangle$$

Pasa a ser

$$\langle \text{expresion} \rangle ::= \langle \text{expresion_simple} \rangle \langle \text{expresion_prima} \rangle$$
$$\begin{aligned} \langle \text{expresion_prima} \rangle &::= \langle \text{relacion} \rangle \langle \text{expresion_simple} \rangle \\ &| \lambda \end{aligned}$$

$$\begin{aligned} \langle \text{termino} \rangle &::= \langle \text{factor} \rangle \\ &| \langle \text{factor} \rangle \langle \text{lista_terminos} \rangle \end{aligned}$$

Pasa a ser

$$\langle \text{termino} \rangle ::= \langle \text{factor} \rangle \langle \text{termino_prima} \rangle$$
$$\begin{aligned} \langle \text{termino_prima} \rangle &::= \langle \text{lista_terminos} \rangle \\ &| \lambda \end{aligned}$$

$$\begin{aligned} \langle \text{lista_terminos} \rangle &::= \langle \text{operacion} \rangle \langle \text{factor} \rangle \\ &| \langle \text{operacion} \rangle \langle \text{factor} \rangle \langle \text{lista_terminos} \rangle \end{aligned}$$

Pasa a ser

$$\langle \text{lista_terminos} \rangle ::= \langle \text{operacion} \rangle \langle \text{factor} \rangle \langle \text{lista_terminos_prima} \rangle$$
$$\begin{aligned} \langle \text{lista_terminos_prima} \rangle &::= \langle \text{lista_terminos} \rangle \\ &| \lambda \end{aligned}$$

$$\begin{aligned} \langle \text{factor} \rangle &::= \langle \text{ident} \rangle \\ &| \langle \text{numero} \rangle \\ &| \text{'true'} \\ &| \text{'false'} \\ &| \langle \text{llamada_funcion} \rangle \\ &| \text{'('} \langle \text{expresion} \rangle \text{'}' \\ &| \text{'not'} \langle \text{factor} \rangle \end{aligned}$$
$$\begin{aligned} \langle \text{llamada_funcion} \rangle &::= \langle \text{ident} \rangle \\ &| \langle \text{ident} \rangle \text{'('} \langle \text{lista_expresion} \rangle \text{'}' \end{aligned}$$

Pasa a ser

$$\begin{aligned}\langle factor \rangle &::= \langle ident \rangle \langle llamada_funcion \rangle \\ &| \langle numero \rangle \\ &| \text{'true'} \\ &| \text{'false'} \\ &| \text{'('} \langle expresion \rangle \text{'})'} \\ &| \text{'not'} \langle factor \rangle\end{aligned}$$

$$\begin{aligned}\langle llamada_funcion \rangle &::= \text{'('} \langle lista_expresion \rangle \text{'})'} \\ &| \lambda\end{aligned}$$

$$\begin{aligned}\langle ident \rangle &::= \langle letra \rangle \\ &| \langle letra \rangle \langle lista_ident \rangle\end{aligned}$$

Pasa a ser

$$\langle ident \rangle ::= \langle letra \rangle \langle ident_prima \rangle$$

$$\begin{aligned}\langle ident_prima \rangle &::= \langle lista_ident \rangle \\ &| \lambda\end{aligned}$$

$$\begin{aligned}\langle lista_ident \rangle &::= \langle letra \rangle | \langle digito \rangle \\ &| \langle letra \rangle \langle lista_ident \rangle \\ &| \langle digito \rangle \langle lista_ident \rangle\end{aligned}$$

Pasa a ser

$$\begin{aligned}\langle lista_ident \rangle &::= \langle letra \rangle \langle lista_ident_prima \rangle \\ &| \langle digito \rangle \langle lista_ident_prima \rangle\end{aligned}$$

$$\begin{aligned}\langle lista_ident_prima \rangle &::= \langle letra \rangle \langle lista_ident \rangle \\ &| \langle digito \rangle \langle lista_ident \rangle \\ &| \lambda\end{aligned}$$

$$\begin{aligned}\langle numero \rangle &::= \langle digito \rangle \langle numero \rangle \\ &| \langle digito \rangle\end{aligned}$$

Pasa a ser

$$\langle numero \rangle ::= \langle digito \rangle \langle numero_prima \rangle$$

$$\langle numero_prima \rangle ::= \langle digito \rangle \langle numero \rangle | \lambda$$

5 Gramática de tipo LL(1)

Para que una gramática sea de tipo LL(1) la misma debe cumplir tres condiciones:

- No ser ambigua.
- No presentar recursión a izquierda.
- Debe estar factorizada a izquierda.

Nuestra gramática esta cerca de cumplir esas características si no fuera por un caso específico. Cuando se presenta una estructura como la siguiente:

```
...
<IF> (expresion) <THEN> (sentencia)
    <IF> (expresion) <THEN> (sentencia)
<ELSE> (sentencia)
...
// En pascal tanto \t como \n se ignoran. Por lo tanto la expresion
    anterior es igual a:
<IF> (expresion) <THEN> (sentencia) <IF> (expresion) <THEN> (
    sentencia) <ELSE> (sentencia)
```

En este caso el $\langle ELSE \rangle$ puede generarse tanto como de la derivación del primer $\langle IF \rangle$ como del segundo. Esto lo resolvemos forzando a cualquier $\langle ELSE \rangle$ que se encuentre a corresponderse al $\langle IF \rangle$ más cercano a izquierda, cumpliendo con la interpretación estándar del Lenguaje Pascal.

6 Gramática final

$$\langle statement \rangle ::= \text{'program'} \langle ident \rangle \text{' ; ' } \langle bloque \rangle \text{' . '}$$
$$\begin{aligned} \langle bloque \rangle &::= \text{'var'} \langle sentencia_de_tipos \rangle \langle lista_subprogramas \rangle \langle cuerpo \rangle \\ &\quad | \langle lista_subprogramas \rangle \langle cuerpo \rangle \\ &\quad | \langle cuerpo \rangle \end{aligned}$$
$$\langle sentencia_de_tipos \rangle ::= \langle declaración_de_var \rangle \text{' ; ' } \langle sentencia_de_tipos_prima \rangle$$
$$\begin{aligned} \langle sentencia_de_tipos_prima \rangle &::= \langle sentencia_de_tipos \rangle \\ &\quad | \lambda \end{aligned}$$
$$\langle declaración_de_var \rangle ::= \langle lista_var \rangle \text{' : ' } \langle tipo \rangle$$
$$\langle lista_var \rangle ::= \langle ident \rangle \text{' , ' } \langle lista_var_prima \rangle$$
$$\begin{aligned} \langle lista_var_prima \rangle &::= \langle lista_var \rangle \\ &\quad | \lambda \end{aligned}$$

$\langle \text{tipo} \rangle ::= \text{'integer'} \mid \text{'boolean'}$

$\langle \text{lista_subprogramas} \rangle ::= \langle \text{subprograma} \rangle \langle \text{lista_subprogramas_prima} \rangle$

$\langle \text{lista_subprogramas_prima} \rangle ::= \text{';' } \langle \text{lista_subprogramas} \rangle$
 $\mid \lambda$

$\langle \text{subprograma} \rangle ::= \langle \text{funcion} \rangle \mid \langle \text{procedimiento} \rangle$

$\langle \text{funcion} \rangle ::= \text{'function'} \langle \text{ident} \rangle \langle \text{funcion_prima} \rangle$

$\langle \text{funcion_prima} \rangle ::= \text{'(' } \langle \text{param_formales} \rangle \text{')' ' : ' } \langle \text{tipo} \rangle \text{' ; ' } \langle \text{bloque} \rangle$
 $\mid \text{' : ' } \langle \text{tipo} \rangle \text{' ; ' } \langle \text{bloque} \rangle$

$\langle \text{procedimiento} \rangle ::= \text{'procedure'} \langle \text{ident} \rangle \langle \text{procedimiento_prima} \rangle$

$\langle \text{procedimiento_prima} \rangle ::= \text{'(' } \langle \text{param_formales} \rangle \text{')' ' ; ' } \langle \text{bloque} \rangle$
 $\mid \text{' ; ' } \langle \text{bloque} \rangle$

$\langle \text{param_formales} \rangle ::= \langle \text{lista_ident} \rangle \text{' : ' } \langle \text{tipo} \rangle \langle \text{param_formales_prima} \rangle$

$\langle \text{param_formales_prima} \rangle ::= \text{' ; ' } \langle \text{param_formales} \rangle$
 $\mid \lambda$

$\langle \text{cuerpo} \rangle ::= \text{'begin'} \langle \text{lista_sentencias} \rangle \text{'end'}$

$\langle \text{lista_sentencias} \rangle ::= \langle \text{sentencia} \rangle \langle \text{lista_sentencias_prima} \rangle$

$\langle \text{lista_sentencias_prima} \rangle ::= \text{' ; ' } \langle \text{lista_sentencias} \rangle$
 $\mid \lambda$

$\langle \text{sentencia} \rangle ::= \langle \text{ident} \rangle \langle \text{sentencia_prima} \rangle$
 $\mid \langle \text{alternativa} \rangle$
 $\mid \langle \text{repetitiva} \rangle$
 $\mid \langle \text{cuerpo} \rangle$
 $\mid \langle \text{lectura} \rangle$
 $\mid \langle \text{escritura} \rangle$

$\langle \text{sentencia_prima} \rangle ::= \text{' := ' } \langle \text{expresion} \rangle$
 $\mid \text{' (' } \langle \text{lista_expresion} \rangle \text{') '}$
 $\mid \lambda$

$\langle \text{alternativa} \rangle ::= \text{'if'} \langle \text{expresion} \rangle \text{'then'} \langle \text{sentencia} \rangle \langle \text{alternativa_prima} \rangle$

$\langle \text{alternativa_prima} \rangle ::= \text{'else'} \langle \text{sentencia} \rangle$
 $\quad \mid \lambda$

$\langle \text{repetitiva} \rangle ::= \text{'while'} \langle \text{expresion} \rangle \text{'do'} \langle \text{sentencia} \rangle$

$\langle \text{lectura} \rangle ::= \text{'read'} \text{'('} \langle \text{ident} \rangle \text{'})'}$

$\langle \text{escritura} \rangle ::= \text{'write'} \text{'('} \langle \text{expresion} \rangle \text{'})'}$

$\langle \text{lista_expresion} \rangle ::= \langle \text{expresion} \rangle \langle \text{lista_expresion_prima} \rangle$

$\langle \text{lista_expresion_prima} \rangle ::= \text{' ,' } \langle \text{lista_expresion} \rangle$
 $\quad \mid \lambda$

$\langle \text{expresion} \rangle ::= \langle \text{expresion_simple} \rangle \langle \text{expresion_prima} \rangle$

$\langle \text{expresion_prima} \rangle ::= \langle \text{relacion} \rangle \langle \text{expresion_simple} \rangle$
 $\quad \mid \lambda$

$\langle \text{relacion} \rangle ::= \text{'='} \mid \text{'<>'} \mid \text{'<'} \mid \text{'<='} \mid \text{'>'} \mid \text{'>='}$

$\langle \text{expresion_simple} \rangle ::= \langle \text{termino} \rangle \langle \text{expresion_simple_prima} \rangle$
 $\quad \mid \langle \text{signo} \rangle \langle \text{termino} \rangle \langle \text{expresion_simple_prima} \rangle$

$\langle \text{expresion_simple_prima} \rangle ::= \langle \text{lista_expresion_simple} \rangle \langle \text{expresion_simple_prima} \rangle \mid \lambda$

$\langle \text{lista_expresion_simple} \rangle ::= \text{'or'} \langle \text{termino} \rangle$
 $\quad \mid \langle \text{signo} \rangle \langle \text{termino} \rangle$

$\langle \text{signo} \rangle ::= \text{'+'} \mid \text{'-'}$

$\langle \text{termino} \rangle ::= \langle \text{factor} \rangle \langle \text{termino_prima} \rangle$

$\langle \text{termino_prima} \rangle ::= \langle \text{lista_terminos} \rangle$
 $\quad \mid \lambda$

$\langle \text{lista_terminos} \rangle ::= \langle \text{operacion} \rangle \langle \text{factor} \rangle \langle \text{lista_terminos_prima} \rangle$

$\langle \text{lista_terminos_prima} \rangle ::= \langle \text{lista_terminos} \rangle$
 $\quad \mid \lambda$

$\langle \text{operacion} \rangle ::= \text{'*'} \mid \text{'/'} \mid \text{'and'}$

$$\begin{aligned} \langle factor \rangle &::= \langle ident \rangle \langle llamada_funcion \rangle \\ &| \langle numero \rangle \\ &| 'true' \\ &| 'false' \\ &| '(' \langle expresion \rangle ')' \\ &| 'not' \langle factor \rangle \end{aligned}$$

$$\begin{aligned} \langle llamada_funcion \rangle &::= '(' \langle lista_expresion \rangle ')' \\ &| \lambda \end{aligned}$$

$$\langle ident \rangle ::= \langle letra \rangle \langle ident_prima \rangle$$

$$\begin{aligned} \langle ident_prima \rangle &::= \langle lista_ident \rangle \\ &| \lambda \end{aligned}$$

$$\begin{aligned} \langle lista_ident \rangle &::= \langle letra \rangle \langle lista_ident_prima \rangle \\ &| \langle digito \rangle \langle lista_ident_prima \rangle \end{aligned}$$

$$\begin{aligned} \langle lista_ident_prima \rangle &::= \langle letra \rangle \langle lista_ident \rangle \\ &| \langle digito \rangle \langle lista_ident \rangle \\ &| \lambda \end{aligned}$$

$$\begin{aligned} \langle letra \rangle &::= 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i' | 'j' | 'k' | 'l' | 'm' | 'n' | 'o' | 'p' | 'q' | 'r' | 's' | \\ &'t' | 'u' | 'v' | 'w' | 'x' | 'y' | 'z' \end{aligned}$$

$$\langle numero \rangle ::= \langle digito \rangle \langle numero_prima \rangle$$

$$\langle numero_prima \rangle ::= \langle digito \rangle \langle numero \rangle | \lambda$$

$$\langle digito \rangle ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'$$

7 Analizador Sintáctico Descendente Recursivo Predictivo

```
// --- RECORDATORIO: definicion de un token en el Analizador Lexico
---
class Token:
def __init__(self, name, value, line_number):
    self.name = name          # Categoria: 'ID', 'NUM', 'IF', '
    OP_REL', etc.
    self.value = value        # Atributo: puntero a tabla, valor
    especifico o vacio.
    self.line_number = line_number # Linea de origen en el codigo
    fuente.
```

```

// --- M dulo Principal y Funciones Auxiliares ---

MODULO principal() {
    // Inicializamos pidiendo el primer token a la clase
    LexiAnalyzer
    lookahead = lex.get_next_token();
    statement();
}

MODULO match(nombreEsperado) {
    // Evaluamos el atributo 'name' del objeto Token
    SI lookahead.name == nombreEsperado ENTONCES
        lookahead = lex.get_next_token();
    SINO
        reportarErrorSintactico ( Se esperaba el token:  +
            nombreEsperado);
    FIN SI
}

// --- Procedimientos de la Gram tica ---

MODULO statement() {
    match('PROGRAM');
    ident();
    match('PUNTO_COMA');
    bloque();
    match('PUNTO');
}

MODULO bloque() {
    SI lookahead.name == 'VAR' ENTONCES
        match('VAR');
        sentencia_de_tipos();
        lista_subprogramas();
        cuerpo();
    SINO SI lookahead.name == 'FUNCTION' O lookahead.name == '
    PROCEDURE' ENTONCES
        lista_subprogramas();
        cuerpo();
    SINO SI lookahead.name == 'BEGIN' ENTONCES
        cuerpo();
    SINO
        reportarErrorSintactico ( Se esperaba VAR, FUNCTION,
            PROCEDURE o BEGIN );
    FIN SI
}

MODULO sentencia_de_tipos() {

```

```

    declaracion_de_var();
    match('PUNTO_COMA');
    sentencia_de_tipos_prima();
}

MODULO sentencia_de_tipos_prima() {
    // Si el siguiente token es un identificador, es una nueva
    declaraci n
    SI lookahead.name == 'ID' ENTONCES
        sentencia_de_tipos();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO declaracion_de_var() {
    lista_var();
    match('DOS_PUNTOS');
    tipo();
}

MODULO lista_var() {
    ident();
    match('COMA');
    lista_var_prima();
}

MODULO lista_var_prima() {
    SI lookahead.name == 'ID' ENTONCES
        lista_var();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO tipo() {
    SI lookahead.name == 'INTEGER' ENTONCES
        match('INTEGER');
    SINO SI lookahead.name == 'BOOLEAN' ENTONCES
        match('BOOLEAN');
    SINO
        reportarErrorSintactico( Tipo de dato inv lido );
    FIN SI
}

MODULO lista_subprogramas() {
    subprograma();
    lista_subprogramas_prima();
}

```

```

MODULO lista_subprogramas_prima() {
    SI lookahead.name == 'PUNTO_COMA' ENTONCES
        match('PUNTO_COMA');
        lista_subprogramas();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO subprograma() {
    SI lookahead.name == 'FUNCTION' ENTONCES
        funcion();
    SINO SI lookahead.name == 'PROCEDURE' ENTONCES
        procedimiento();
    SINO
        reportarErrorSintactico ( Se esperaba FUNCTION o PROCEDURE );
    FIN SI
}

MODULO funcion() {
    match('FUNCTION');
    ident();
    funcion_prima();
}

MODULO funcion_prima() {
    SI lookahead.name == 'PAR_ABRE' ENTONCES
        match('PAR_ABRE');
        param_formales();
        match('PAR_CIERRA');
        match('DOS_PUNTOS');
        tipo();
        match('PUNTO_COMA');
        bloque();
    SINO SI lookahead.name == 'DOS_PUNTOS' ENTONCES
        match('DOS_PUNTOS');
        tipo();
        match('PUNTO_COMA');
        bloque();
    SINO
        reportarErrorSintactico ( Error en declaraci n de funci n )
        ;
    FIN SI
}

MODULO procedimiento() {
    match('PROCEDURE');
    ident();
    procedimiento_prima();
}

```

```

}

MODULO procedimiento_prima() {
    SI lookahead.name == 'PAR_ABRE' ENTONCES
        match('PAR_ABRE');
        param_formales();
        match('PAR_CIERRA');
        match('PUNTO_COMA');
        bloque();
    SINO SI lookahead.name == 'PUNTO_COMA' ENTONCES
        match('PUNTO_COMA');
        bloque();
    SINO
        reportarErrorSintactico( Error en declaraci n de
                                procedimiento );
    FIN SI
}

MODULO param_formales() {
    lista_ident();
    match('DOS_PUNTOS');
    tipo();
    param_formales_prima();
}

MODULO param_formales_prima() {
    SI lookahead.name == 'PUNTO_COMA' ENTONCES
        match('PUNTO_COMA');
        param_formales();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO cuerpo() {
    match('BEGIN');
    lista_sentencias();
    match('END');
}

MODULO lista_sentencias() {
    sentencia();
    lista_sentencias_prima();
}

MODULO lista_sentencias_prima() {
    SI lookahead.name == 'PUNTO_COMA' ENTONCES
        match('PUNTO_COMA');
        lista_sentencias();

```

```

        // SINO: derivaci n en      (no hace nada)
        FIN SI
    }

MODULO sentencia() {
    SI lookahead.name == 'ID' ENTONCES
        ident();
        sentencia_prima();
    SINO SI lookahead.name == 'IF' ENTONCES
        alternativa();
    SINO SI lookahead.name == 'WHILE' ENTONCES
        repetitiva();
    SINO SI lookahead.name == 'BEGIN' ENTONCES
        cuerpo();
    SINO SI lookahead.name == 'READ' ENTONCES
        lectura();
    SINO SI lookahead.name == 'WRITE' ENTONCES
        escritura();
    SINO
        reportarErrorSintactico ( Sentencia inv lida );
    FIN SI
}

MODULO sentencia_prima() {
    SI lookahead.name == 'OP_ASIG' ENTONCES
        match('OP_ASIG');
        expresion();
    SINO SI lookahead.name == 'PAR_ABRE' ENTONCES
        match('PAR_ABRE');
        lista_expresion();
        match('PAR_CIERRA');
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO alternativa() {
    match('IF');
    expresion();
    match('THEN');
    sentencia();
    alternativa_prima();
}

MODULO alternativa_prima() {
    // Se fuerza a matchear el else con el if m s cercano
    SI lookahead.name == 'ELSE' ENTONCES
        match('ELSE');
        sentencia();
}

```

```

        // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO repetitiva() {
    match('WHILE');
    expresion();
    match('DO');
    sentencia();
}

MODULO lectura() {
    match('READ');
    match('PAR_ABRE');
    ident();
    match('PAR_CIERRA');
}

MODULO escritura() {
    match('WRITE');
    match('PAR_ABRE');
    expresion();
    match('PAR_CIERRA');
}

MODULO lista_expresion() {
    expresion();
    lista_expresion_prima();
}

MODULO lista_expresion_prima() {
    SI lookahead.name == 'COMA' ENTONCES
        match('COMA');
        lista_expresion();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO expresion() {
    expresion_simple();
    expresion_prima();
}

MODULO expresion_prima() {
    SI lookahead.name == 'OP_REL' ENTONCES
        relacion();
        expresion_simple();
    // SINO: derivaci n en      (no hace nada)

```

```

    FIN SI
}

MODULO relacion() {
    SI lookahead.name == 'OP_REL' ENTONCES
        match('OP_REL ');
    SINO
        reportarErrorSintactico ( Se esperaba un operador relacional
        );
    FIN SI
}

MODULO expresion_simple() {
    SI lookahead.name == 'OP_ARIT' Y (lookahead.value == 'ADD' O
    lookahead.value == 'SUB') ENTONCES
        signo();
        termino();
        expresion_simple_prima();
    SINO SI es_inicio_de_factor() ENTONCES
        termino();
        expresion_simple_prima();
    SINO
        reportarErrorSintactico ( Error en expresi n simple );
    FIN SI
}

MODULO expresion_simple_prima() {
    SI lookahead.name == 'OR' O (lookahead.name == 'OP_ARIT' Y (
    lookahead.value == 'ADD' O lookahead.value == 'SUB'))
    ENTONCES
        lista_expresion_simple();
        expresion_simple_prima();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO lista_expresion_simple() {
    SI lookahead.name == 'OR' ENTONCES
        match('OR ');
        termino();
    SINO SI lookahead.name == 'OP_ARIT' Y (lookahead.value == 'ADD'
    O lookahead.value == 'SUB') ENTONCES
        signo();
        termino();
    SINO
        reportarErrorSintactico ( Error en operador de expresi n );
    FIN SI
}

```

```

MODULO signo() {
    SI lookahead.name == 'OP_ARIT' Y lookahead.value == 'ADD'
        ENTONCES
            match('OP_ARIT');
    SINO SI lookahead.name == 'OP_ARIT' Y lookahead.value == 'SUB'
        ENTONCES
            match('OP_ARIT');
    SINO
        reportarErrorSintactico ( Se esperaba un signo (+ o -) );
    FIN SI
}

MODULO termino() {
    factor();
    termino_prima();
}

MODULO termino_prima() {
    SI lookahead.name == 'AND' O (lookahead.name == 'OP_ARIT' Y (
        lookahead.value == 'MUL' O lookahead.value == 'DIV'))
        ENTONCES
            lista_terminos();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO lista_terminos() {
    operacion();
    factor();
    lista_terminos_prima();
}

MODULO lista_terminos_prima() {
    SI lookahead.name == 'AND' O (lookahead.name == 'OP_ARIT' Y (
        lookahead.value == 'MUL' O lookahead.value == 'DIV'))
        ENTONCES
            lista_terminos();
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

MODULO operacion() {
    SI lookahead.name == 'OP_ARIT' Y lookahead.value == 'MUL'
        ENTONCES
            match('OP_ARIT');
    SINO SI lookahead.name == 'OP_ARIT' Y lookahead.value == 'DIV'
        ENTONCES

```

```

        match('OP_ARIT');
    SINO SI lookahead.name == 'AND' ENTONCES
        match('AND');
    SINO
        reportarErrorSintactico(Se esperaba '*', '/' o 'and');
    FIN SI
}

MODULO factor() {
    SI lookahead.name == 'ID' ENTONCES
        ident();
        llamada_funcion();
    SINO SI lookahead.name == 'NUM' ENTONCES
        numero();
    SINO SI lookahead.name == 'TRUE' ENTONCES
        match('TRUE');
    SINO SI lookahead.name == 'FALSE' ENTONCES
        match('FALSE');
    SINO SI lookahead.name == 'PAR_ABRE' ENTONCES
        match('PAR_ABRE');
        expresion();
        match('PAR_CIERRA');
    SINO SI lookahead.name == 'NOT' ENTONCES
        match('NOT');
        factor();
    SINO
        reportarErrorSintactico(Error en factor);
    FIN SI
}

MODULO llamada_funcion() {
    SI lookahead.name == 'PAR_ABRE' ENTONCES
        match('PAR_ABRE');
        lista_expresion();
        match('PAR_CIERRA');
    // SINO: derivaci n en      (no hace nada)
    FIN SI
}

// --- Integraci n con el Analizador L xico (Laboratorio 2 y 3)
---

MODULO ident() {
    match('ID');
}

MODULO numero() {
    match('NUM');
}

```

```

}

FUNCION es_inicio_de_factor() {
    RETORNAR (lookahead.name == 'ID' O lookahead.name == 'NUM' O
        lookahead.name == 'TRUE' O lookahead.name == 'FALSE' O
        lookahead.name == 'PAR_ABRE' O lookahead.name == 'NOT');
}

```

8 Problemas encontrados

Durante el desarrollo surgieron varios desafíos técnicos:

- **Conflicto de Identificadores:** Fue complejo manejar el hecho de que tanto las variables como las funciones y procedimientos comienzan con un token ID. Esto nos obligó a refactorizar las reglas de <sentencia> y <factor> para postergar la decisión hasta encontrar un operador de asignación (:=) o un paréntesis de apertura.
- **Ambigüedad del Else:** Aunque la gramática fue factorizada, el conflicto lógico del `else` persistía. La solución no fue puramente gramatical, sino que requirió una decisión de diseño en la lógica del procedimiento recursivo.

9 Conclusión

En conclusión, la realización de este trabajo permitió comprender que la construcción de un compilador no es solo una tarea de programación, sino de diseño formal. Además, nos permitió realizar la transformación de una gramática que cumple con EBNF a una LL1, lo cual fue la tarea mas importante para asegurar la correctitud del analizador y nos permitió interiorizar los conceptos abarcados a lo largo de la materia.